

# Paper

“‘latex id="x8bz2j"”

## 1 Summary

Bài báo *Context Features Are Cheap: Rank-Aware Decomposition for Efficient Feature Interaction in Recommender Systems* nghiên cứu bài toán tối ưu chi phí inference trong các hệ recommender công nghiệp. Trong pipeline recommender phổ biến, hệ thống thường có hai tầng: retrieval và ranking. Tầng retrieval lấy ra một tập nhỏ gồm  $N$  candidate items từ corpus rất lớn, sau đó tầng ranking dùng một deep model để score từng candidate theo cùng một user và cùng một context. Vì một request có nhiều candidate nhưng context không đổi, nếu model xử lý context lặp lại cho từng candidate thì sẽ tạo ra rất nhiều phép tính dư thừa.

Vấn đề chính mà bài báo đặt ra là nhiều ranking architectures hiện đại như DLRM, DeepFM, DCNv2 hoặc attention-based CTR models thường kết hợp context features và target/item features rất sớm trong forward pass. Context features có thể gồm user profile, browsing history, session state, page context; target features là các đặc trưng riêng của từng item candidate. Khi context được broadcast sang chiều candidate ngay từ đầu, mọi phép tính chỉ phụ thuộc context cũng bị lặp lại  $N$  lần. Điều này gây lãng phí lớn trong production serving.

Ý tưởng trung tâm của bài báo là *rank-aware decomposition*. Ở đây, “rank” không phải là thứ hạng recommendation, mà là tensor rank theo chiều candidate. Context features là rank-2 tensor vì giống nhau cho mọi candidate trong request, còn target features là rank-3 tensor vì thay đổi theo từng candidate. Nếu giữ context và target tách riêng càng lâu càng tốt, các phép tính chỉ phụ thuộc context có thể được tính một lần cho mỗi request thay vì  $N$  lần cho mỗi candidate.

Bài báo xây dựng nguyên lý đại số chung: mọi phép toán tuyến tính hoặc song tuyến tính trên input được chia thành context block và target block đều có thể phân rã chính xác theo block. Với phép toán tuyến tính:

$$x = [x_c; x_t]$$

$$Wx = W_c x_c + W_t x_t$$

Trong đó  $x_c$  là context part và  $x_t$  là target part. Vì  $x_c$  không đổi giữa các candidates, thành phần  $W_c x_c$  chỉ cần tính một lần rồi broadcast lại.

Với phép toán song tuyến tính:

$$x^T W y$$

khi:

$$x = [x_c; x_t], \quad y = [y_c; y_t]$$

ta có thể phân rã:

$$x^T W y = x_c^T W_{cc} y_c + x_c^T W_{ct} y_t + x_t^T W_{tc} y_c + x_t^T W_{tt} y_t$$

Trong đó thành phần  $x_c^T W_{cc} y_c$  chỉ phụ thuộc context và có thể tính một lần cho mỗi request. Đây là nguyên lý chung được áp dụng cho FM interaction, fully connected layer, DCNv2 cross layer và self-attention.

Trường hợp đầu tiên là *Factorization Machine* (FM) interaction. Trong các model kiểu DLRM hoặc DeepFM, interaction layer thường tính tất cả pairwise dot products giữa các feature embeddings:

$$FM(e) = \{\langle e_i, e_j \rangle\}_{1 \leq i < j \leq K+M}$$

Trong đó  $K$  là số context fields,  $M$  là số target fields, và  $D$  là embedding dimension. Nếu tính theo cách thông thường, toàn bộ  $(K + M)$  features được broadcast theo  $N$  candidates, nên cả context-context interactions cũng bị tính lại  $N$  lần.

Bài báo phân rã FM thành hai phần:

$$FM(e) = [FM_{ctx}(c); FM_{tgt}(c, T)]$$

Trong đó:

$$FM_{ctx}(c) = \{\langle e_{c_i}, e_{c_j} \rangle\}_{1 \leq i < j \leq K}$$

là toàn bộ context-context interactions, được tính một lần cho mỗi request. Phần còn lại:

$$FM_{tgt}(c, T^{(n)}) = \{\langle e_{t_i}^{(n)}, e_{c_k} \rangle\}_{i,k} \cup \{\langle e_{t_i}^{(n)}, e_{t_j}^{(n)} \rangle\}_{i < j}$$

bao gồm context-target và target-target interactions, vẫn phải tính theo từng candidate.

FLOP savings của FM decomposition được viết:

$$\Delta_{FM} = (N - 1) \binom{K}{2} D$$

Điểm quan trọng là savings tăng theo:

$$O(K^2)$$

vì context-context pairs có số lượng  $\binom{K}{2}$ . Do đó, khi hệ thống thêm nhiều context features như user history, session signals hoặc page features, rank-aware decomposition càng có lợi.

Trường hợp thứ hai là *Fully Connected* (FC) layer. Sau interaction layer, recommender model thường đưa vector interaction vào một MLP. Với first FC layer:

$$h = \sigma([x_c; x_t]W + b)$$

có thể phân rã thành:

$$h = \sigma(x_c W_c + x_t W_t + b)$$

Thành phần  $x_c W_c$  được tính một lần ở rank-2, sau đó broadcast sang  $N$  candidates và cộng với phần target-dependent  $x_t W_t$ . Bài báo gọi implementation này là *RankSplitDense*. Savings của FC layer là:

$$\Delta_{FC} = (N - 1)|x_c|U$$

trong đó  $U$  là số units của FC layer.

Nếu FC layer nằm sau FM decomposition, thì  $x_c = FM_{ctx}(c)$  có kích thước:

$$|x_c| = \binom{K}{2}$$

Do đó:

$$\Delta_{FC} = (N - 1)\binom{K}{2}U$$

Nghĩa là FC savings cũng tăng theo  $O(K^2)$  khi input context part đến từ FM context-context interactions. Đây là lý do FM + FC rank-aware decomposition tạo ra lợi ích lớn hơn nhiều so với tối ưu MLP thuần, vốn chỉ tiết kiệm tuyến tính theo số context features.

Trường hợp thứ ba là *DCNv2 cross layer*. DCNv2 có dạng:

$$x_{l+1} = x_0 \odot (W_l x_l + b_l) + x_l$$

Với input được tách thành context và target, phép nhân ma trận có thể phân rã:

$$W_l x_l = \begin{bmatrix} W_{cc} & W_{ct} \\ W_{tc} & W_{tt} \end{bmatrix} \begin{bmatrix} x_l^c \\ x_l^t \end{bmatrix}$$

Ở layer đầu tiên, các thành phần phụ thuộc context như  $W_{cc}x_0^c$  và  $W_{tc}x_0^c$  có thể tính một lần. Tuy nhiên, sau layer đầu, context và target đã bị trộn với nhau trong output, nên phần được gọi là context không còn thuần context nữa. Vì vậy, identity-equivalent rank-aware decomposition chỉ áp dụng hiệu quả ở first cross layer của DCNv2.

Trường hợp thứ tư là *self-attention*. Trong attention-based interaction models như

AutoInt, các projection  $Q$ ,  $K$ ,  $V$  đều là phép tuyến tính nên có thể phân rã theo context và target:

$$W_Q E = W_{Q,c} E_c + W_{Q,t} E_t$$

Tương tự cho  $K$  và  $V$ . Attention score matrix cũng có block structure:

$$\begin{bmatrix} Q_c K_c^T & Q_c K_t^T \\ Q_t K_c^T & Q_t K_t^T \end{bmatrix}$$

Trong đó block  $Q_c K_c^T$  chỉ phụ thuộc context và có thể tính một lần. Tuy nhiên, softmax normalization trộn các block trong denominator, nên việc tối ưu attention cần cẩn thận hơn. Bài báo cho rằng có thể dùng kỹ thuật kiểu block-wise log-sum-exp tương tự FlashAttention để kết hợp precomputed context block với per-candidate target block. Dù vậy, cũng giống DCNv2, attention layer sau khi output đã mix context và target thì decomposition chỉ còn áp dụng ở first layer nếu giữ nguyên kiến trúc gốc.

Từ giới hạn này, bài báo đề xuất một hướng kiến trúc mới: *rank-aware architecture design*. Nguyên tắc là không cho target stream làm ô nhiễm context stream. Context stream chỉ cập nhật bằng context-context dynamics, còn context-target interactions được nạp vào target stream. Bài báo gọi đây là *asymmetric loading principle*.

Cụ thể, paper đề xuất *rDCN*, một biến thể rank-aware của DCNv2. rDCN duy trì hai state riêng:

$$c_l$$

là context stream rank-2, và:

$$T_l$$

là target stream rank-3. Update của target stream:

$$T_{l+1} = T_0 \odot (W_l^{ct} c_l \oplus W_l^t T_l + b_l^t) + T_l$$

Update của context stream:

$$c_{l+1} = c_0 \odot (W_l^c c_l + b_l^c) + c_l$$

Trong đó  $W_l^{ct} c_l$  được tính một lần rồi broadcast sang chiều candidate, còn  $W_l^t T_l$  vẫn tính theo từng candidate. Vì target không được map ngược vào context output, context stream vẫn giữ rank-2 qua toàn bộ depth. Nhờ vậy, savings có thể tích lũy qua nhiều layers thay vì chỉ ở first layer.

So với DCNv2 chuẩn, rDCN loại bỏ block  $W_{ct}$ , tức block target-to-context coupling. Baseline DCNv2 có chi phí mỗi layer xấp xỉ:

$$\Theta(N(D_c + D_t)^2)$$

Trong khi rDCN giảm thành:

$$\Theta(D_c^2) + \Theta(N(D_t D_c + D_t^2))$$

Điểm quan trọng là phần context-driven computation được đưa ra ngoài vòng lặp  $N$  candidates.

Bài báo cũng phác thảo một biến thể rank-aware attention. Context stream dùng self-attention chỉ trên context fields. Target stream dùng target queries nhưng attention đến cả context keys/values và target keys/values. Context keys và values được tính ở rank-2 một lần, còn target part tính theo candidate. Cách thiết kế này tương tự rDCN: context stream không bị target làm ô nhiễm, còn target stream vẫn có thể tận dụng context.

Về evaluation, paper kiểm tra ba nhóm kết quả: FLOP analysis, synthetic throughput characterization và production deployment. Trong offline evaluation, identity-equivalent decomposition cho kết quả prediction giống vanilla model gần như tuyệt đối, đúng như kỳ vọng vì đây chỉ là biến đổi đại số. Với DLRM-style model, FM-only rank-aware giảm interaction FLOPs 87%, còn FM+FC rank-aware giảm tổng FLOPs 73% với logloss gần như không đổi.

Với DCN-style model, identity-equivalent DCNv2+FC giảm tổng FLOPs khoảng 20%. Trong khi đó, rDCN+FC giảm tổng FLOPs khoảng 67% và vẫn giữ logloss trong vùng training noise. Khi bỏ parallel context stream trong rDCN, logloss xấu đi rõ hơn, cho thấy context-context dynamics vẫn là thành phần quan trọng và không nên loại bỏ.

Synthetic throughput experiment cho thấy rank-aware decomposition đặc biệt có lợi khi tăng số context features  $K$ . Khi giữ  $M = 4$  và tăng  $K$  từ 8 lên 24, vanilla RPS giảm mạnh từ 1.00 xuống 0.34, còn rank-aware RPS giảm nhẹ hơn từ 1.13 xuống 0.69. Rank-aware advantage tăng từ +13% ở  $K = 8$  lên +105% ở  $K = 24$ . Ngược lại, khi giữ  $K = 8$  và tăng target features  $M$  từ 4 lên 24, hai đường throughput giảm gần song song, vì target-side compute vẫn phải tính theo từng candidate.

Kết quả production là phần thuyết phục nhất của paper. Khi triển khai trên DLRM-style ranker trong production platform của Taboola, FM rank-aware tăng per-pod throughput 25%. Khi thêm FC rank-aware, throughput tăng 87.5%, tương đương giảm 47% peak pod count, trong khi prediction vẫn giống baseline. Latency cũng giảm mạnh: p50 giảm 30% và p99 giảm 33%.

## 2 Conclusion

Hướng đi đã làm của bài báo là tối ưu inference cho multi-target recommender ranking bằng cách khai thác sự khác biệt giữa context features và target features. Thay vì broadcast context sang mọi candidate ngay từ đầu, bài báo giữ context ở rank-2 và target ở rank-3, rồi phân rã các phép toán tuyến tính/song tuyến tính để tính context-only

computation một lần cho mỗi request. Đây là hướng tối ưu không làm thay đổi prediction đối với các kiến trúc hiện có khi áp dụng identity-equivalent decomposition.

Đóng góp kỹ thuật quan trọng nhất là *rank-aware decomposition principle*. Nguyên lý này áp dụng cho nhiều interaction mechanisms phổ biến: FM pairwise products, FC projection layers, DCNv2 first cross layer và self-attention first layer. Với các model DLRM-style, lợi ích đặc biệt lớn vì context-context FM interactions có số lượng  $\binom{K}{2}$ , làm savings tăng theo bình phương số context features.

Kết quả thực nghiệm xác nhận rằng context features trở nên “rẻ” hơn nhiều so với target features sau decomposition. Khi tăng context features, vanilla model bị giảm throughput mạnh, còn rank-aware model suy giảm chậm hơn đáng kể. Khi tăng target features, lợi ích ít hơn vì target-dependent computation vẫn phải thực hiện cho từng candidate. Điều này dẫn đến một trade-off thiết kế mới: trong các rank-aware recommender, việc thêm context features có thể ít tốn kém hơn nhiều so với thêm target features.

Bài báo cũng chỉ ra giới hạn của identity-equivalent decomposition: với DCNv2 và attention, sau layer đầu tiên context và target đã bị trộn, nên không thể tiếp tục precompute context-only computation ở các layer sau. Để vượt qua giới hạn này, paper đề xuất *rDCN*, một kiến trúc mới duy trì rank discipline qua nhiều layer bằng cách tách context stream và target stream. rDCN không hoàn toàn identity-equivalent với DCNv2, nhưng đạt accuracy tương đương trong offline evaluation với tổng FLOPs thấp hơn 67%.

Hướng kế thừa từ bài báo có thể gồm nhiều nhánh. Thứ nhất, cần triển khai và đánh giá online rDCN trong production để kiểm tra liệu FLOP savings offline có chuyển thành throughput/latency gains thực tế hay không. Thứ hai, cần hiện thực hóa rank-aware attention trong các recommender architecture có attention-based feature interaction. Thứ ba, cần mở rộng rank-aware decomposition cho các mô hình early-fusion hoặc target-attention như DIN, ETA, TWIN, TransAct, nơi user/history representation đã phụ thuộc target ngay từ layer đầu.

Ngoài ra, paper này rất phù hợp để kế thừa trong hướng thiết kế recommender system có nhiều context signals. Các hệ recommender hiện đại ngày càng dùng nhiều context như session, device, page, user history, temporal context hoặc location. Nếu dùng kiến trúc vanilla, thêm context có thể làm serving cost tăng mạnh. Nhưng với rank-aware decomposition, nhiều context-context computations có thể được tính một lần cho mỗi request. Vì vậy, hướng mở rộng tự nhiên là xây dựng các ranking model vừa giàu context hơn, vừa giữ được chi phí inference thấp bằng rank-aware architecture design. ““